

Nama : Muh Khalifman Ali

NIM : 245150407111079

Matkul : Pengembangan Aplikasi Mobile - D

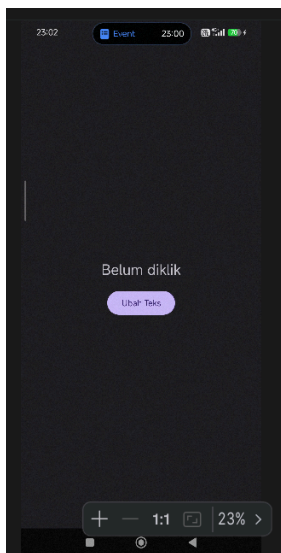
Tugas Modul 3 Praktikum

I. Project compose vs XML (imperatif & deklaratif)

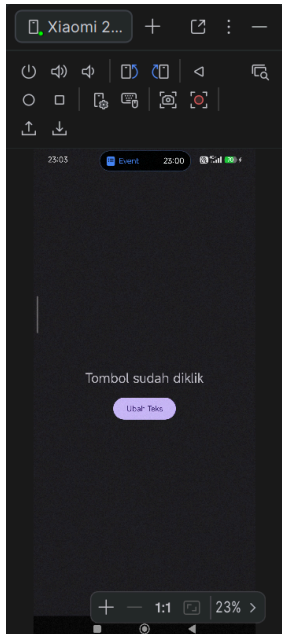
A. Project A (xml/view - imperatif)

1. Jalankan aplikasi. Pastikan teks berubah setelah tombol diklik.

Sebelum tombol di klik:



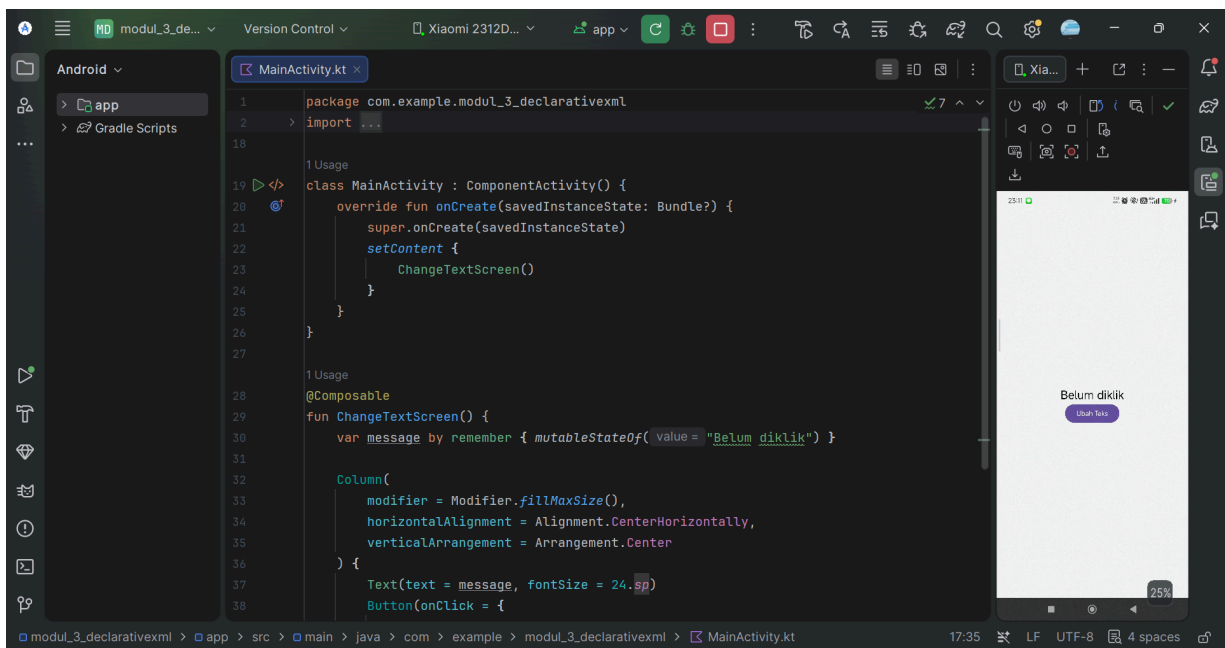
Sesudah diklik:



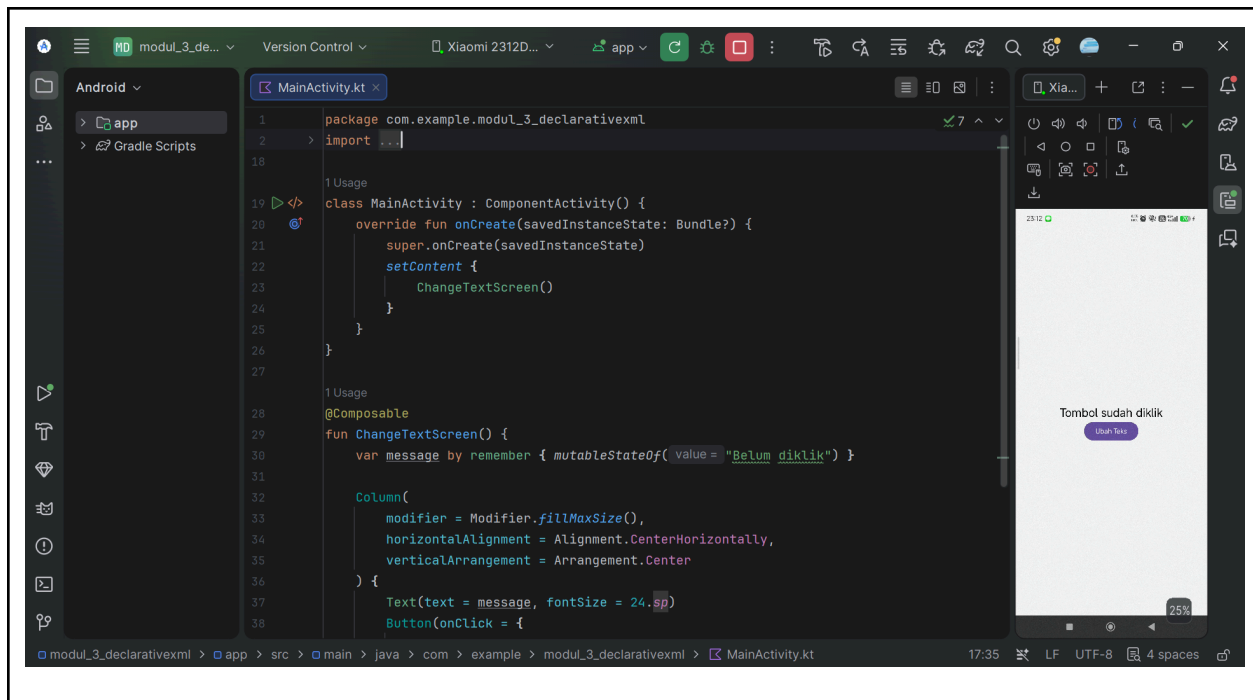
B. Project B Jetpack compose (deklaratif)

1. Jalankan aplikasi sebelum dan sesudah teks berubah atau tidak

Sebelum:



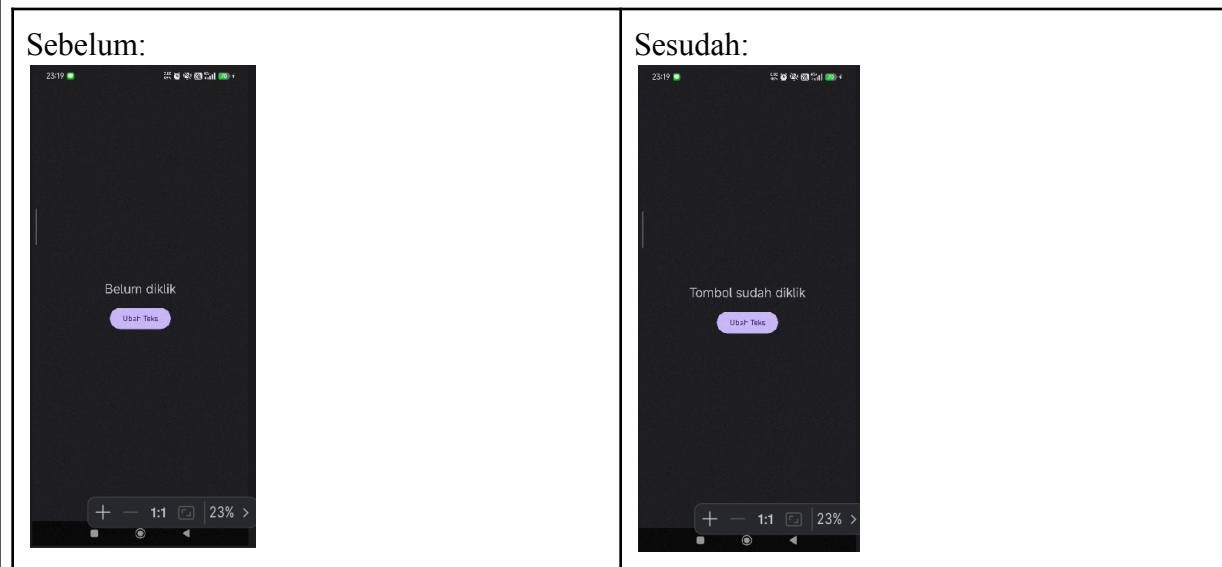
Sesudah:



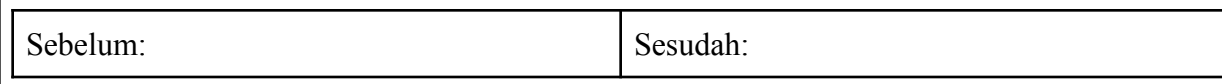
C. Aktivitas Percobaan

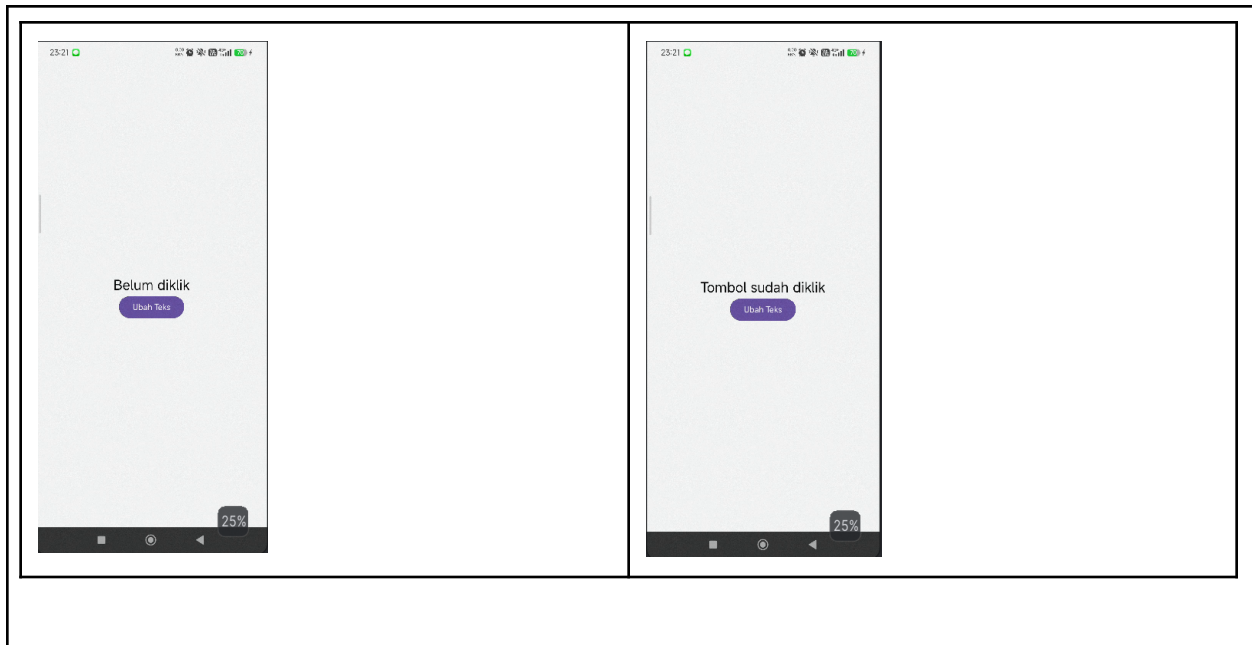
1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

XML:



Jetpack compose:





2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Objek yang diubah langsung adalah instance TextView bernama tvMessage

Baris kode: `tvMessage.text = "Tombol sudah diklik"`

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Nilai yang diubah adalah variabel state message

Baris kode: `message = "Tombol sudah diklik"`

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Karena Compose menggunakan arsitektur deklaratif di mana UI merepresentasikan fungsi dari state yaitu ($UI = f(state)$). Developer tidak memanipulasi hirarki objek View secara manual, tetapi hanya memperbarui data dari statenya, lalu framework compose akan otomatis melakukan recomposition pada komponen UI

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

event (klik tombol) -> view (btnChange akan mengtrigger `setOnClickListener`) -> perubahan tampilan (lalu memanggil mutasi properti `tvMessage.text` secara langsung).

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Event (klik tombol onClick) -> (nilai message diperbarui) -> recomposition (Compose akan mendeteksi ada perubahan state lalu menggambar ulang fungsi dari text) -> terjadi perubahan tampilan.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Jika diganti nilai variabel message di memori akan berubah saat tombol diklik, tetapi tampilan di layar tidak berubah.

Penyebabnya karena variabel biasa tidak didaftarkan sebagai state yang bisa diamati dengan compose, akhirnya sistem tidak mengetahui terjadi perubahan data dan tidak mentrigger proses recomposition

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Imperatif: fokusnya ke bagaimana cara mengubah UI dengan manual/step by step (cari dahulu komponennya, lalu ganti propertinya).

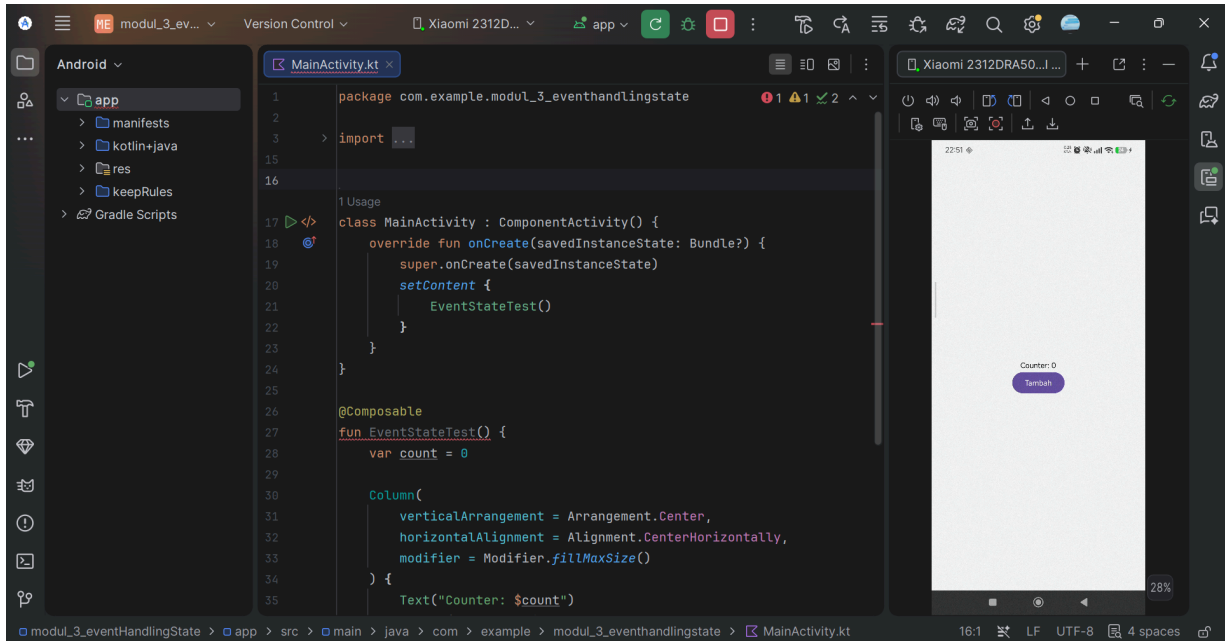
Deklaratif: sedangkan deklaratif fokusnya ke wujud UI berdasarkan kondisi data/state saat ini (UI adalah cerminan dari state; jika mengubah datanya, maka tampilannya akan otomatis menyesuaikan).

II. Event handling state

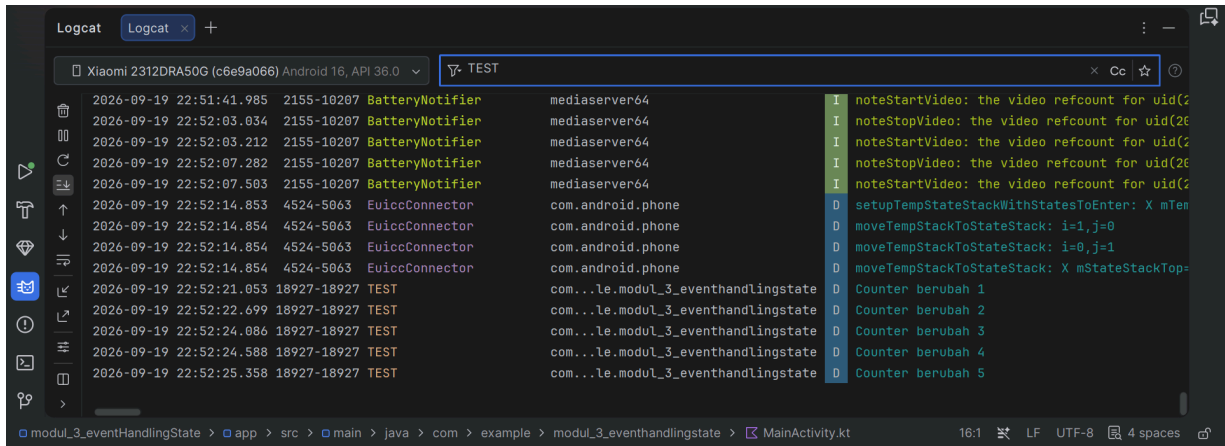
A. Aktivitas percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

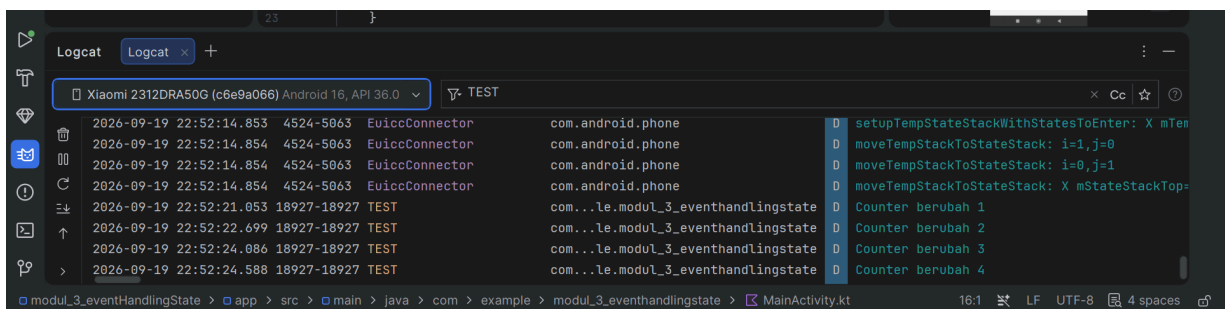
Run:



Logcat:



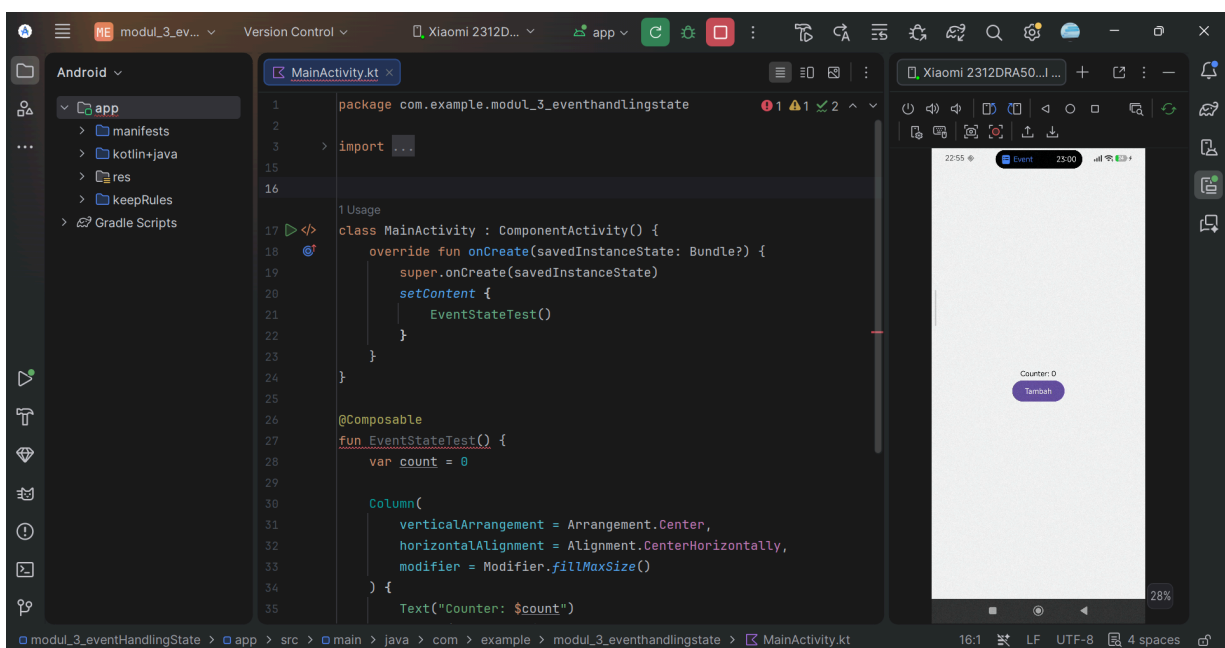
2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?



Logcat terus bertambah dengan counter:

Counter berubah 1, counter berubah 2, Counter berubah 3, dst..

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?



Tampilan angka pada (Counter: angka) tetap Counter: 0 dan tidak berubah di layar.

4. Apa yang menyebabkan hal tersebut terjadi?

Karena count hanya di deklarasi menjadi variabel default saja (var count = 0) yang dimana variabel biasa tidak dapat meng-trigger compose untuk mengubah angka atau mengubah antarmuka layar

5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.

```
Kotlin
@Composable
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }

    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter: $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah $count")
        }) {
            Text("Tambah")
        }
    }
}
```

6. Ubah kode program pada bagian fungsi Composable EventStateTest dari yang sebelumnya:

`var count = 0`

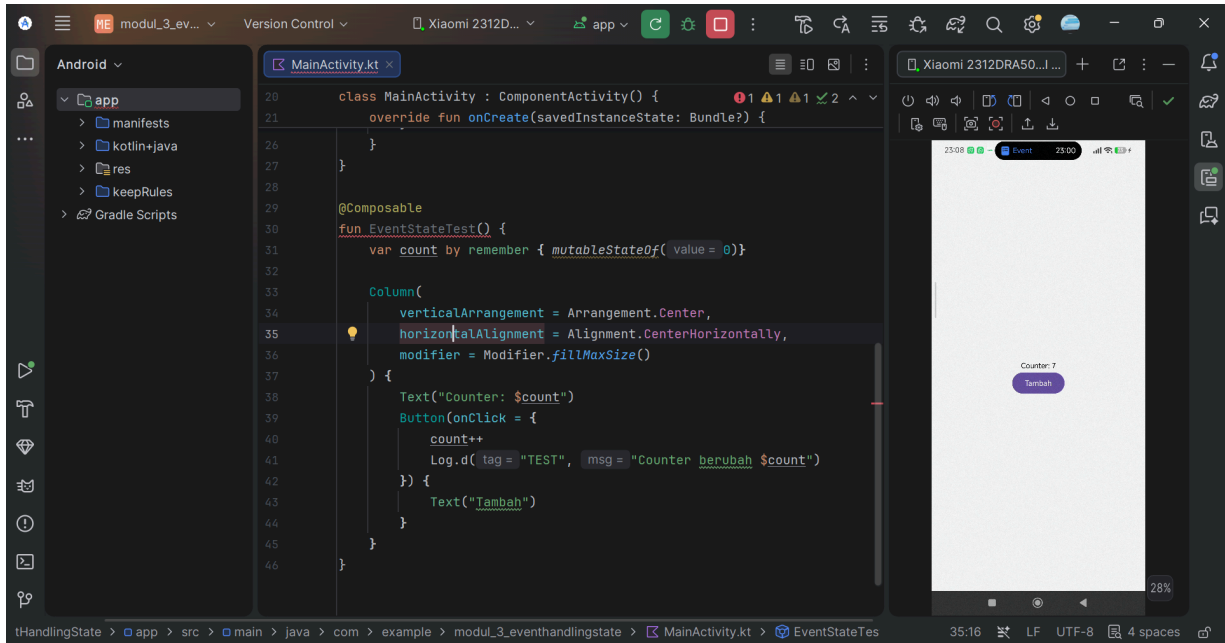
menjadi:

`var count by remember { mutableStateOf(0) }`

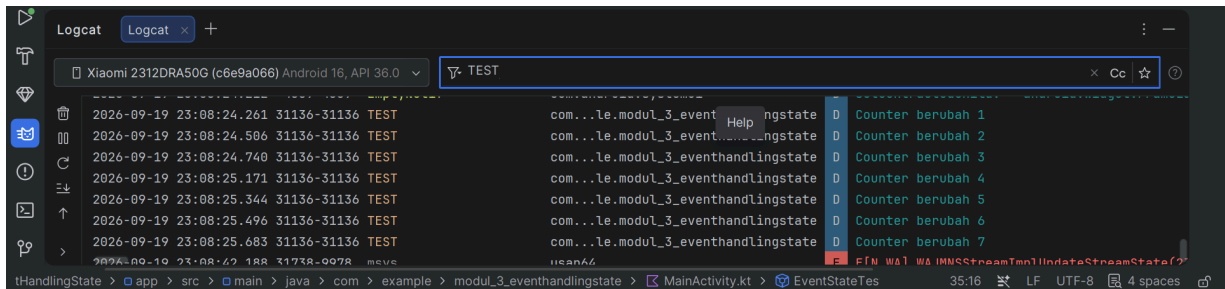
```
var count by remember { mutableStateOf( value = 0) }
```

7. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

Run:



Logcat:



8. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Output pada logcat terjadi perubahan nilai counter: Counter berubah 1, Counter berubah 2, dst.

9. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Sekarang teks di layar HP otomatis berubah dan diperbarui menjadi nilai angka yang menyesuaikan dengan klik-an (counter: 1, counter: 2, dst....)

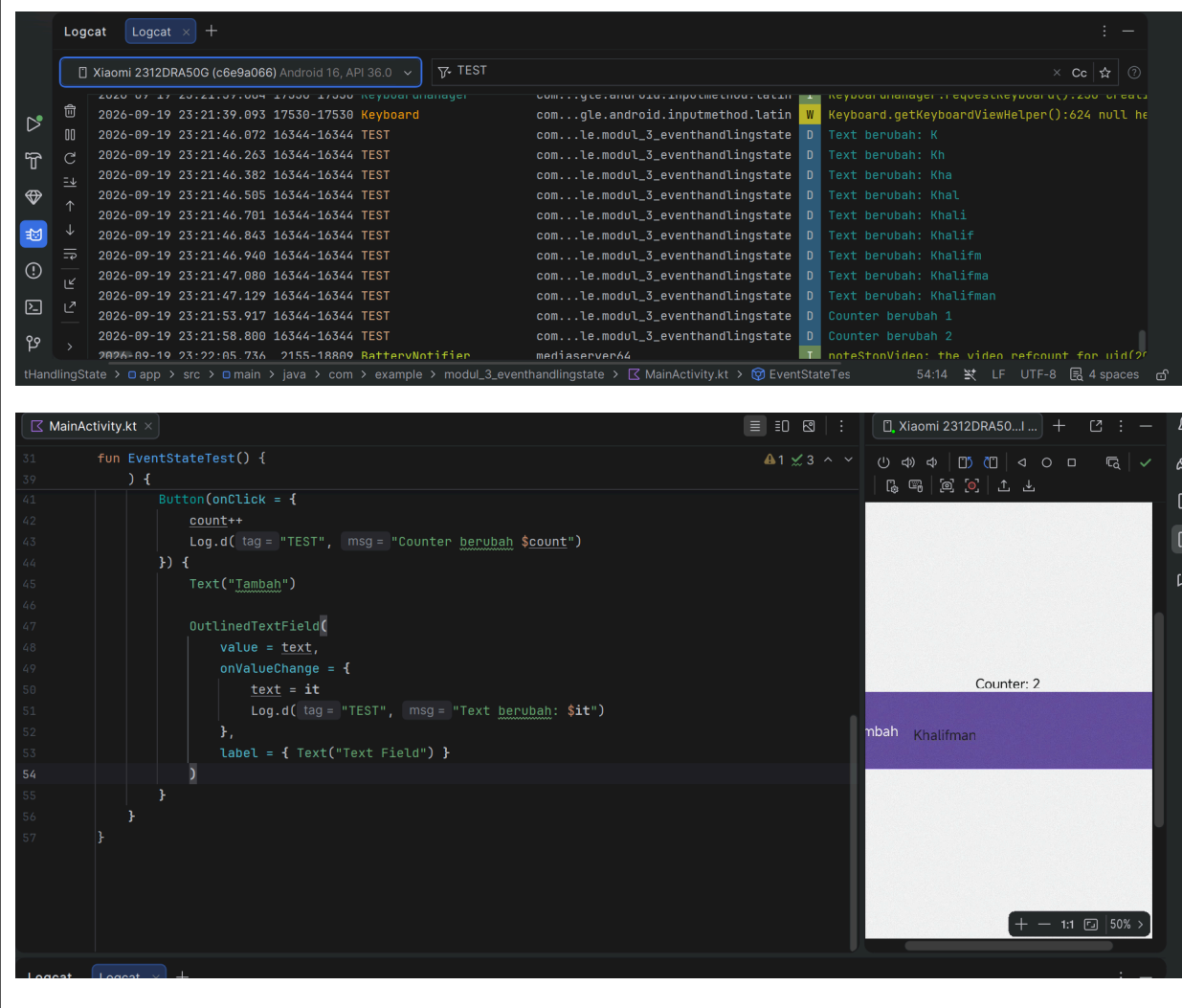
10. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Menggunakan `mutableStateOf(0)` yang menjadikan variabel `count` menjadi observable state. Atau fungsi `remember` ini membuat nilai state tersimpan dalam proses composition. Dan saat nilai state berubah, `compose` akan mendeteksi perubahan lalu mengtrigger recomposition untuk memperbarui tampilan di layar.

11. Apa yang dimaksud recomposition pada Compose?

Recomposition adalah proses yang dimana Jetpack Compose akan mengeksekusi ulang fungsi `@Composable` yang terpengaruh dengan perubahan data state, akhirnya tampilan UI akan diperbarui sesuai dengan state terbaru.

12. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?



Logcat mengeluarkan output log Text berubah: <karakter yang diketik> dengan real-time setiap ada karakter yang ditambah.

13. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Karakter yang diketik langsung muncul di dalam kotak OutlinedTextField dengan real time.

14. Apa yang menyebabkan hal tersebut terjadi?

Karakter baru dari event `onValueChange` memperbarui state text (`text = it`), lalu mentrigger recomposition akhirnya OutlinedTextField menampilkan nilai `value = text` baru

15. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

1. State integer count (menyimpan angka counter)
2. State string text (menyimpan isi teks dari box input)

16. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

1. Event `onClick` pada Button.
2. Event `onValueChange` pada OutlinedTextField

17. Kapan event - event tersebut dijalankan?

`onClick`: akan dijalankan ketika user menekan tombol "Tambah".

`onValueChange`: akan dijalankan setiap kali ada perubahan input keyboard (mengetik atau menghapus karakter) di dalam OutlinedTextField.

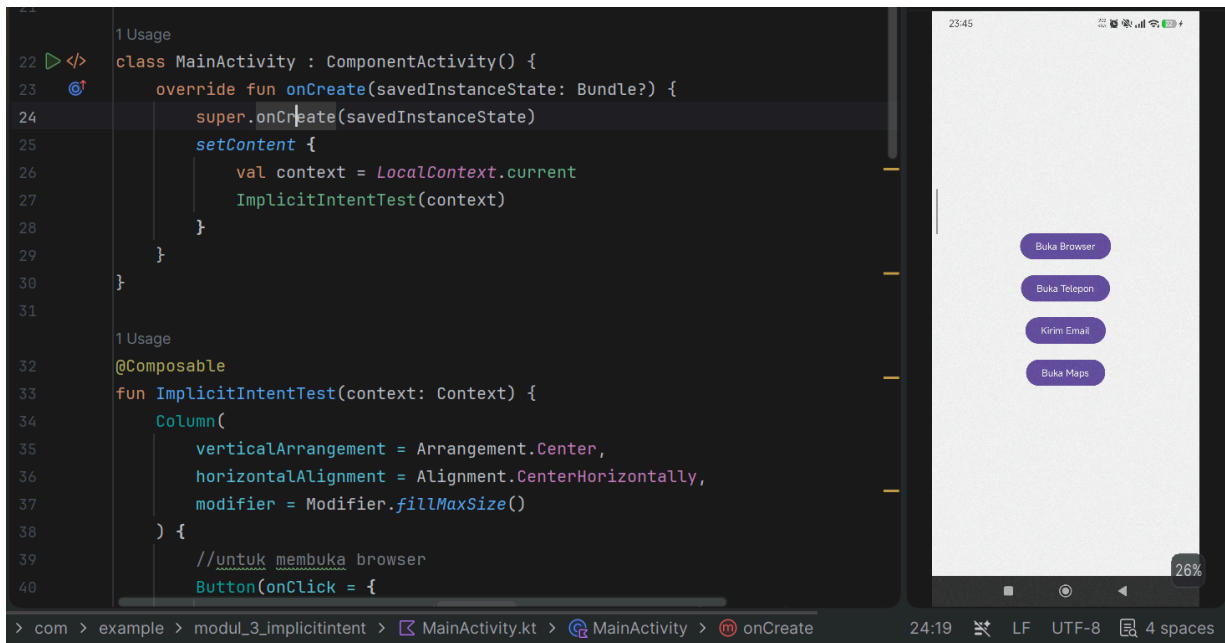
III. Implicit intent

A. Aktivitas percobaan

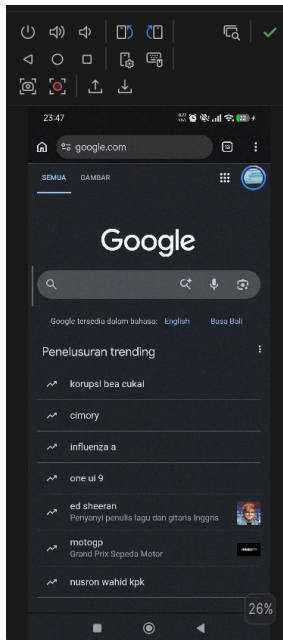
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:

(Klik Buka Browser, Klik Buka Telepon, Klik Kirim Email, Klik Buka Maps)

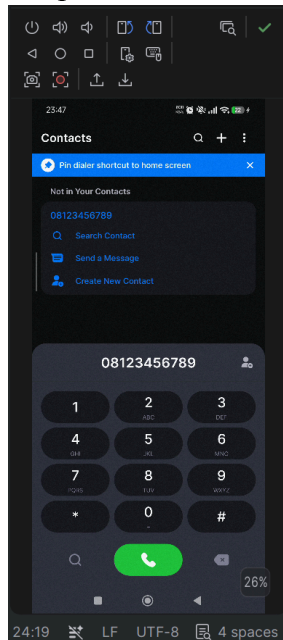
Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya



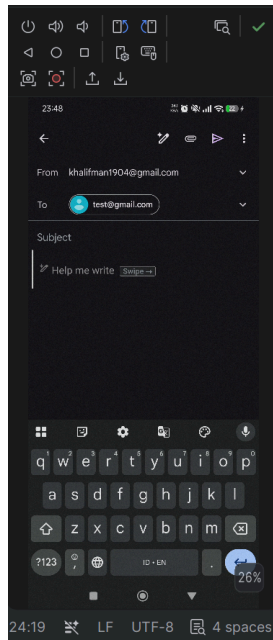
Browser:



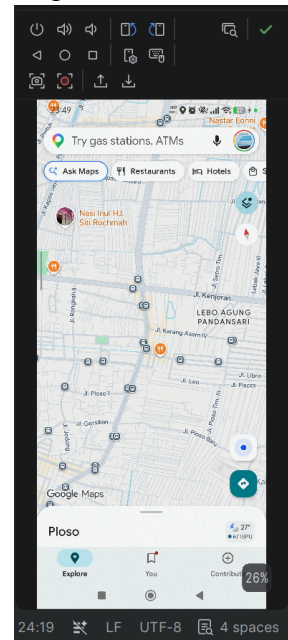
Telepon:



Email:



Maps:



--	--	--	--

2. Pada kode yang digunakan, terdapat beberapa action seperti:

(Intent.ACTION_VIEW, Intent.ACTION_DIAL, Intent.ACTION_SENDTO)

Apa perbedaan fungsi dari masing-masing action intent tersebut?

- Intent.ACTION_VIEW: berguna untuk menampilkan data ke pengguna (contohnya membuka URL di browser atau menampilkan lokasi di maps)
- Intent.ACTION_DIAL: digunakan untuk membuka aplikasi telepon dan bisa pula mengisi nomor tujuan tapi tidak otomatis melakukan telepon.
- Intent.ACTION_SENDTO: digunakan untuk mengirimkan pesan atau data ke tujuan yang lebih spesifik, biasanya digunakan untuk membuka aplikasi email berdasarkan skema URI mailto:

3. Pada kode yang digunakan, terdapat beberapa data seperti:

data = Uri.parse("mailto:test@gmail.com")

data = Uri.parse("https://www.google.com")

data = Uri.parse("geo:-7.250445,112.768845")

Apa fungsi data pada intent ?

Berfungsi untuk menentukan objek data (beserta skema skenario penanganannya) yang ingin diproses oleh Intent.

Objek Uri memberikan petunjuk jenis data (seperti https:// untuk web, tel: untuk telepon, mailto: untuk email, dan geo: untuk peta)

4. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Sistem Android mencocokkan aksi yang ingin dilakukan dengan data (atau format data yang dikirim) dengan daftar Intent Filter yang terdaftar pada manifest aplikasi yang ada di HP.

Jika ada aplikasi yang terdaftar cocok menangani kombinasi action + data tersebut (misal: ACTION_VIEW + https://) android akan langsung membuka atau menampilkan pilihan aplikasi jika ada lebih dari satu aplikasi yang bisa mendeteksi intent tersebut.